Module 5 — Advanced Topics in Cloud Computing

Complete Notes

5.1 Energy Efficiency, Market-Based Management, Federated Clouds & Third-Party Services

Energy Efficiency in Clouds

Data Center Power Consumption

- Data Center energy demand doubles every 5–6 years.
- This results in large amounts of CO_2 produced by burning fossil fuels.
- Need to reduce the energy used with minimal performance impact.

Ref: Efficient Resource Management for Cloud Computing Environments — Younge, von Laszewski, Wang, Lopez-Alarcon, Carithers.

Datacenters are New Polluters

- U.S. data centers use more electricity than they need. In 2017, it took 34 power plants (each ~500 MW) to power all U.S. data centers.
- By 2020, the nation needed another 17 similarly-sized power plants to meet projected data center energy demand as economic activity becomes increasingly digital.
- Lawrence Berkeley National Laboratory predicts data center demand will grow from 176 TWh in 2023 (~4.4% of total U.S. electricity consumption) to between 325–580 TWh (6.7–12.0%) by 2028.
- In some parts of the country, AI-driven energy demand is outpacing available capacity, driving companies to delay projects, contract power directly from private producers, and/or install multiple inefficient reciprocating generators using natural gas.
- Any increase in the use of fossil fuels to generate electricity will result in an increase in carbon emissions.

Fossil Fuels

What are Fossil Fuels?

- Formed over millions of years from fossils (remains) of dead animals and plants buried under dirt and rock.
- Heat from inside the earth and pressure from dirt and rock changes these fossils into oil, natural gas, and coal.

Three major forms of fossil fuels:

- Coal
- Oil
- Natural gas

All three were formed millions of years ago during the time of the dinosaurs.

Why are Fossil Fuels Important?

- Due to rapid growth in AI and data centers, electricity demand is surging.
- In the U.S., utilities are:
  - Extending life of coal plants
  - Building new natural gas plants
- It takes millions of years to make or "renew" more fossil fuels — so they are called nonrenewable fuels.
- We are currently using fuels made more than 65 million years ago.
- Once this fuel is gone, it is gone.

Typical Data Center Energy Consumption

Reference: Dzmitry Kliazovich, University of Luxembourg.

Impact of Cloud DC on Environment

- Data centers are not only expensive to maintain, but also unfriendly to the environment.
- Carbon emissions due to Data Centers worldwide are now more than both Argentina and the Netherlands combined.
- High energy costs and huge carbon footprints are incurred:
  - Due to the massive amount of electricity needed to power the servers.
  - And the energy needed to cool the numerous servers hosted in these data centers.
- According to the McKinsey report "Revolutionizing Data Center Energy Efficiency", a typical datacenter consumes as much energy as 25,000 households.

Warehouse-Scale Data-Center Design

Data-Center Construction Requirements

- Most data centers are built with commercially available components.
- An off-the-shelf server consists of:
  - A number of processor sockets,
  - Each with a multicore CPU and its internal cache hierarchy,
  - Local shared and coherent DRAM,
  - A number of directly attached disk drives.

- ("Off-the-shelf" = not designed or made to order, but taken from existing stock or supplies.)
- DRAM and disk resources within the rack are accessible through first-level rack switches; all resources in all racks are accessible via a cluster-level switch.
- Consider a data center:
    - Built with 2,000 servers,
    - Each with 8 GB of DRAM and four 1 TB disk drives.
    - Each group of 40 servers is connected through a 1 Gbps link to a rack-level switch.
    - Each rack switch has an additional eight 1 Gbps ports used for connecting the rack to the cluster-level switch.

Cooling System of a Data-Center Room

- The data-center room has raised floors for hiding cables, power lines, and cooling supplies.
- The raised floor has a steel grid resting on stanchions about 2–4 ft above the concrete floor. (Stanchion = frame/fixture used to support another object.)
- The under-floor area is often used to route power cables to racks, but its primary use is to distribute cool air to the server rack.
- The CRAC (Computer Room Air Conditioning) unit pressurizes the raised-floor plenum by blowing cold air into the plenum.
- The cold air escapes from the plenum through perforated tiles placed in front of server racks.
- Racks are arranged in long aisles that alternate between cold aisles and hot aisles to avoid mixing hot and cold air.
- The hot air produced by the servers circulates back to the intakes of the CRAC units, which cool it and exhaust the cool air into the raised-floor plenum again.
- Typically, the incoming coolant is at 12–14°C and the warm coolant returns to a chiller.

Green Computing

Advanced scheduling schemes to reduce energy consumption:

- Power aware
- Thermal aware

Data center designs to reduce power usage:

- Cooling systems
- Rack design

Performance ↔ Energy Efficiency

- There is a need to shift focus from optimizing data center resource management for pure performance alone to optimizing for energy efficiency while maintaining high service-level performance.

Energy Efficiency in Clouds

- Cloud service providers need to adopt measures to ensure that their profit margins are not dramatically reduced due to high energy costs.
- Alternatives:
  - Solar + wind energy are expanding rapidly.
  - Many data center companies (like hyperscalers) are investing in clean energy.
- According to Amazon's estimate, energy-related costs of its datacenters amount to 42% of the total budget — including direct power consumption and the cooling infrastructure amortized over a 15-year period.
- Companies such as Google, Microsoft, and Yahoo! are building large datacenters in barren desert land surrounding the Columbia River (USA) to exploit cheap hydroelectric power.
- There is increasing pressure from governments worldwide to reduce carbon footprints, which significantly impact climate change.
- To address these concerns, leading IT vendors formed a global consortium called The Green Grid to promote energy efficiency for datacenters and minimize their environmental impact.
- The Green Grid is a nonprofit, industry consortium of end-users, policy-makers, technology providers, facility architects, and utility companies collaborating to improve the resource efficiency of data centers (https://www.thegreengrid.org/).
- Green cloud computing is envisioned to achieve not only efficient processing and utilization of computing infrastructure but also minimize energy consumption — essential for ensuring sustainable future growth of cloud computing.
- Cloud resources need to be allocated not only to satisfy QoS requirements specified by users via Service-Level Agreements (SLAs) but also to reduce energy usage.

WUI v1 Scoring Calculator

- The Water Usage Impact (WUI) v1 Metric is a proxy based on the environmental impact of water consumed by a data center, in relationship to the Water Stress Level where the data center is located.
- WUI measures the environmental impact of water used by a data center, considering how scarce water is in that location.
- WUI shifts thinking from "How much water do we use?" → "How harmful is that usage?"
- The calculator provided by The Green Grid takes:
  - Water consumption data
  - Location (to assign stress factor)
- Outputs:
  - WUI score (impact level)
- Used by:

- Data center designers
- Sustainability analysts

Coolant Distribution Unit (CDU) v2 Sizing Tool

- Designed to size a Coolant Distribution Unit (CDU) for a direct-to-chip liquid cooling system.
- Calculates the necessary fluid temperature, total flow rate, and total pressure drop across cold plates in a cooling loop.
- Serves as a preliminary guide to facilitate better understanding and communication between buyers and sellers, helping bridge the knowledge gap in selecting the appropriate CDU for a server.

Energy Consumption of Unused Servers

- To run a server farm (data center) a company has to spend a huge amount of money for hardware, software, operational support, and energy every year.
- Companies should thoroughly identify whether their installed server farm is appropriately utilized.
- On average, one-sixth (≈15%) of the full-time servers in a company are left powered on without being actively used (idling) on a daily basis.
- With ~44 million servers in the world, ~4.7 million servers are not doing any useful work.
- The potential savings from turning off these servers are large.
- IT departments need to analyze their servers and find unused or underutilized servers.

Reducing Energy in Active Servers

- Apply appropriate techniques to decrease energy consumption in active distributed systems with negligible influence on their performance.
- Power management issues in distributed computing platforms can be categorized into four layers:

  - Application layer
  - Middleware layer
  - Resource layer
  - Network layer

Application Layer

- An application's energy consumption depends strongly on:

  - Number of instructions needed to execute the application,
  - Number of transactions with the storage unit (or memory).

- These two factors (compute and storage) are correlated and they affect completion time.
- By introducing energy-aware applications, the challenge is to design applications without hurting performance.
- First step: explore the relationship between performance and energy consumption.

Middleware Layer

- Bridge between the Application Layer and the Resource Layer.
- Provides:
  - Resource broker
  - Communication service
  - Task analyzer
  - Task scheduler
  - Security access
  - Reliability control
  - Information service
- Necessity of a new cost function covering both:
  - Makespan (the execution time of a set of tasks), and
  - Energy consumption.

Resource Layer

- Consists of a wide range of resources including:
  - Computing nodes,
  - Storage units.
- Interacts with hardware devices and the operating system.
- Several mechanisms have been developed for more efficient power management of hardware and operating systems.
- Hardware Approaches (popular methods incorporated into recent computer hardware systems):
  - Dynamic Power Management (DPM)
  - Dynamic Voltage-Frequency Scaling (DVFS)

Resource Layer — DPM

- Dynamic Power Management (DPM): hardware devices, such as the CPU, can switch from idle mode to one or more lower power modes.

Resource Layer — DVFS

- Dynamic Voltage-Frequency Scaling (DVFS): energy savings are achieved because power consumption in CMOS circuits has a direct relationship with frequency and the square of the voltage supply.
- Energy in CMOS circuits is related by parameters:
  - V = voltage
  - Ceff = circuit switching capacity

    - K = a technology-dependent factor
    - Vt = threshold voltage
    - T = execution time of the task under clock frequency f

- Execution time and power consumption are controllable by switching among different frequencies and voltages.
- DVFS enables exploitation of the slack time (idle time) typically incurred by inter-task relationships — by reducing the frequency or voltage during slack time.
- By reducing voltage and frequency, the device's energy consumption can also be reduced.

Network Layer

- Routing and transferring packets and enabling network services to the resource layer are the main responsibilities of the network layer in Distributed Computing Systems.
- Two major challenges:

    - New, energy-efficient routing algorithms need to be developed.
    - New, energy-efficient protocols should be developed against network attacks.

Market-Based Management of Clouds

A Reference Model for MOCC (Market-Oriented Cloud Computing)

- MOCC originated from the coordination of several components:

    - Service consumers,
    - Service providers, and
    - Other entities that make trading between these two groups possible.

- Several components contribute to the realization of the Cloud Exchange and implement its features.

Directory

- The market directory contains a listing of all the published services that are available.
- It contains a mapping between service names and the corresponding vendor (or cloud coordinator) offering them.
- It also provides additional metadata that helps brokers or end users filter services that meet the expected quality of service.

Auctioneer

- Keeps track of running auctions in the marketplace.
- Verifies that auctions for services are properly conducted.
- Prevents malicious market players from performing illegal activities.

Bank

- Takes care of the financial aspect of all operations happening in the virtual marketplace.
- Ensures financial transactions are carried out in a secure and dependable environment.
- Consumers and providers may register with the bank and have one or multiple accounts to perform transactions.

Cloud Exchange (CEx)

- The fundamental component is the virtual marketplace — represented by the Cloud Exchange (CEx).
- Acts as a market maker, bringing service producers and consumers together.

Cloud Broker

- Operates on behalf of consumers.
- Identifies the subset of services that match the customers' requirements in terms of service profile and quality of service.
- Mediates between coordinators and consumers — acquires services from coordinators and subleases them to consumers.
- Brokers can accept requests from many users; users can leverage different brokers.

Cloud Coordinators

- Represent the cloud vendors and publish the services that vendors offer.
- A similar relationship can be considered between coordinators and cloud computing services vendors.
- Take responsibility for publishing and advertising services on behalf of vendors and can gain benefits from reselling services to brokers.

Market-Oriented Cloud Architecture for Data Centers

- As consumers rely on cloud providers to meet more of their computing needs, they will require a specific level of QoS to be maintained by their providers.
- Cloud providers consider and meet the different QoS parameters of each individual consumer as negotiated in specific SLAs.
- Purpose: promote QoS-based resource allocation mechanisms:
    - Clients can benefit from the potential cost reduction of providers,
    - Which could lead to a more competitive market,
    - And thus lower prices.
- Users or brokers (acting on the user's behalf) submit service requests from anywhere in the world to the data center / cloud to be processed.
- They originate the workload that is managed in the cloud datacenter.

SLA Resource Allocator

- Acts as the interface between the data center / cloud service provider and external users / brokers.
- Main responsibility: ensure service requests are satisfied according to the SLA agreed with the user.
- Several components coordinate allocator activities to realize this goal.

Service Request Examiner and Admission Control Module

- When a service request is first submitted, the service request examiner interprets the submitted request for QoS requirements before deciding to accept or reject it.
- Operates in the front-end and filters user / broker requests to accept those that are feasible given the current status of the system and the workload already being processed.
- Accepted requests are allocated and scheduled for execution.

Pricing Mechanism

- Decides how service requests are charged.
- Responsible for charging users according to the SLA they signed.
- Different parameters can be considered in charging users; for instance, IaaS providers commonly charge according to characteristics of the virtual machines requested in terms of memory, disk size, computing capacity, and the time they are used.

Accounting Mechanism

- Maintains the actual usage of resources by requests so the final cost can be computed and charged to users.
- Stores the billing information for each user.
- Constitutes a rich source of information that can be mined to identify usage trends and improve the vendor's service offering.

Dispatcher

- Responsible for the low-level operations required to realize admitted service requests.
- Starts the execution of accepted service requests on allocated VMs.
- In an IaaS scenario, instructs the infrastructure to deploy as many virtual machines as needed to satisfy a user's request.

VM Monitor

- Keeps track of the availability of VMs and their resource entitlements.

Service Request Monitor

- Keeps track of the execution progress of service requests.
- Information collected through it is helpful for analyzing system performance and providing quality feedback about the provider's capability to satisfy requests.

- Multiple VMs can be started and stopped on demand on a single physical machine.

Federated Clouds / Inter Cloud

Federated Clouds — Introduction

- Federated clouds are enablers for MOCC since they provide means for interoperation among different cloud providers.
- They allow building aggregations of clouds that belong to different administrative domains.

Federation

- The creation of an organization that supersedes the decisional and administrative power of single entities and acts as a whole.
- Within a cloud computing context, federation implies that there are agreements between the various cloud providers, allowing them to leverage each other's services in a privileged manner.

Definition (Reuven Cohen, founder and CTO of Enomaly Inc.)

"Cloud federation manages consistency and access controls when two or more independent geographically distinct Clouds share either authentication, files, computing resources, command and control or access to storage resources."

Technologies for Cloud Federations — RESERVOIR

- Even though the concept of cloud federation / InterCloud is still immature, there are supporting technologies that enable the deployment of interoperable clouds — e.g., RESERVOIR.
- RESERVOIR = Resources and Services Virtualization Without Barriers.
- A European research project focused on developing an architecture that supports providers of cloud infrastructures to dynamically partner with each other to extend their capabilities while preserving their administrative autonomy.

RESERVOIR Architecture — Service Manager

- Constitutes the front-end used by service providers to submit service manifests, negotiate pricing, and monitor applications.
- Deploys and provisions VEEs according to the service manifest.
- Monitors and enforces SLA compliance by controlling the capacity of a service application.

RESERVOIR Architecture — Virtual Execution Environment (VEE) Manager

- Core of the RESERVOIR middleware.
- Responsible for the optimal placement of VEEs into VEE hosts according to constraints expressed by the Service Manager.
- Interacts with VEE Managers in other sites to provision additional instances for execution of service applications, or to move VEEs to other sites in case of overload.
- This component realizes cloud federation.

RESERVOIR Architecture — VEE Host (VEEH)

- Interacts with the VEE Manager to put into practice the IT management decisions regarding heterogeneous sets of virtualization platforms.
- Ensures networking among VEEs that belong to the same application.
- Encapsulates all platform-specific management required to expose the used virtualization technology through a standardized interface to the VEE Manager.

RESERVOIR Cloud Deployment

- Defines an infrastructure overlay that spans multiple administrative domains and different geographic locations.
- Each site runs the RESERVOIR software stack.
- Infrastructure providers operate RESERVOIR sites.
- Service providers define service applications.
- First case: the service provider conducts sizing to identify the appropriate number of components required for a given workload condition. The specification is obtained by means of minimal service configuration and a set of elasticity rules used by RESERVOIR.
- Second case: the service provider provides neither a minimal service configuration nor elasticity rules. Sizing is automatically made by the RESERVOIR middleware, which tries to minimize over-provisioning.

InterCloud

- InterCloud is a term often used interchangeably with Cloud Federation.
- Introduced by Cisco for expressing a composition of clouds interconnected by means of open standards to provide a universal environment that leverages cloud computing services.
- Mimicking the Internet (the “network of networks”), InterCloud represents a “Cloud of Clouds.”
- InterCloud expresses the same concept of federating together clouds — in many cases acceptable.
- Some practitioners (e.g., Ellen Rubin — founder and VP of Products at CloudSwitch) prefer different connotations:
    - InterCloud refers mostly to a global vision in which interoperability among cloud providers is governed by standards, creating an open platform where applications can shift workloads and freely compose services from different sources.
    - Cloud Federation is more general and includes ad hoc aggregations between cloud providers based on private agreements and proprietary interfaces.

InterCloud Architecture

- InterCloud is a service-oriented architectural framework for cloud federation that supports utility-driven interconnection of clouds.
- Comprises two main elements:
    - CloudExchange
    - CloudCoordinator

CloudExchange

- The market-making component of the architecture.
- Offers services that allow providers to find each other in order to directly trade cloud assets, and allows parties to register and run auctions.
- In the former case, CloudExchange acts as a directory service for the federation.
- In the latter case, it runs the auction.
- Workflow with Coordinator:
    - 1. Wherever the Coordinator detects that extra resources are required by the datacenter,
    - 2. It triggers the process of discovery of potential providers (by interacting with the cloud federation).
    - 3. Once potential providers are discovered and the preferred one is selected, the Coordinator contacts the remote Coordinator and negotiates.
- Underutilized resources scenario:
    - 1. Similarly, when the Coordinator detects that local resources are underutilized,
    - 2. They can publish an offer for resources in the CloudExchange or look for matches among requirements registered in the Exchange service.

CloudCoordinator

- Has Front-end components:
    - Elements that interact with the federation.
    - Interact with the CloudExchange and with other coordinators.
    - Allow datacenters to announce their offers and requirements.
- Has Back-end components:
    - Elements that interact with the associated datacenter.
    - Allow the Coordinator to learn about the current state of the datacenter to decide whether actions from the federation are required or not.

Third-Party Cloud Services: MetaCDN, SpotCloud

Third-Party Cloud Services — Overview

- One of the key elements of cloud computing is the possibility of composing services that belong to different vendors or integrating them into existing software systems.
- The service-oriented model — basis of cloud computing — facilitates such an approach and provides the opportunity for developing a new class of services called third-party cloud services.
- These are the result of adding value to preexisting cloud computing services, providing customers with a different and more sophisticated service.

- Added value can be created by smartly coordinating existing services or implementing additional features on top of an existing basic service.
- Examples of third-party services:
    - MetaCDN
    - SpotCloud

What is a CDN?

- A Content Delivery Network (CDN) is a network of interconnected servers that speeds up webpage loading for data-heavy applications.
- CDN can stand for Content Delivery Network or Content Distribution Network.
- When a user visits a website, data has to travel across the internet to reach the user's computer. If the user is far from the server, it takes a long time to load large files (video, images).
- Instead, website content is stored on CDN servers geographically closer to the user, reaching their computers much faster.

Why is a CDN Important?

- The primary purpose of a CDN is to reduce latency — the delay in communication created by a network's design.
- Communication traffic between websites (servers) and users (clients) must move over large physical distances.
- Communication is two-way: requests go from client to server and responses come back.
- A CDN improves efficiency by introducing intermediary servers between the client and the website server.
- These CDN servers manage some of the client-server communications. They decrease web traffic to the web server, reduce bandwidth consumption, and improve user experience.
- Analogy: Think of a CDN like an ATM. Having a cash machine on practically every corner makes it fast and efficient to get money — no wait time in long bank lines.
- CDN services were created to solve the problem of network congestion caused by delivering rich web content (graphics, video) over the internet — much like a traffic jam.
- CDNs have grown to include text, graphics, scripts, media files, software downloads, documents, portals, e-commerce, live streaming media, on-demand video streaming media, and social media sites.

Existing CDN Providers

- Akamai is the clear leader in coverage and market share (~80%). Price is prohibitive for SMEs, NGOs, governments, etc.
- Academic CDNs include Coral, Codeen, Globule, etc.

MetaCDN

- MetaCDN provides users with a Content Delivery Network (CDN).

- It implements a software overlay that coordinates the service offerings of different cloud storage vendors and uses them as distributed elastic storage on which user content is stored.
- MetaCDN enables the uniform use of heterogeneous storage clouds as a single, large, distributed content delivery network.
- Advantage: provides a CDN service at accessible costs and enriches the original service offering of existing cloud services with additional functionalities.
- MetaCDN provides users with the high-level services of a CDN for content distribution and interacts with the low-level interfaces of storage clouds to optimally place user content according to expected geography of demand.
- MetaCDN is a pioneering "4th generation" CDN provider. Its patent-pending approach leverages Tier-1 Cloud Storage & CDN suppliers such as Amazon, Microsoft, and Google to offer enterprise-class content delivery, video encoding, and streaming services on a global scale.
- Founded in 2011 from research at the University of Melbourne. Backed by Australian VC firm Starfish Ventures and the University of Melbourne's commercialization arm.
- Idea — "What if we could create a low-cost, high-performance overlay CDN using heterogeneous storage clouds?"
    - Exploit the strengths and coverage footprints of multiple providers.
    - Cherry-pick providers based on customer's QoS needs / cost budget.
    - MetaCDN provides this while hiding the complexity from users.

MetaCDN Architecture

- The MetaCDN interface exposes its services through users and applications via the Web.
- Users interact with a portal, while applications take advantage of the Web services.
- The portal constitutes a more intuitive interface for users with basic requirements.
- The Web service provides access to the full capabilities of MetaCDN and allows for more complex and sophisticated deployment.
- The main operations of MetaCDN are the creation of deployments over storage clouds and their management.

MetaCDN Deployment Options

Four different deployment options can be selected:

- Coverage and performance-optimized deployment:
    - MetaCDN deploys as many replicas as possible to all available locations.
- Direct deployment:
    - MetaCDN allows the selection of the deployment regions for the content and matches the selected regions with the supported providers serving those areas.
- Cost-optimized deployment:

    - MetaCDN deploys as many replicas in the locations identified by the deployment request.
    - The available storage transfer allowance and budget will be used to deploy the replicas and keep them active for as long as possible.

- QoS-optimized deployment:

    - MetaCDN selects providers that can better match the QoS requirements attached to the deployment, such as average response time and throughput from a particular location.

MetaCDN Components

Three particularly important components:

- MetaCDN Manager:

    - Responsible for ensuring that all content deployments meet the expected QoS.
    - Supported by the Monitor.

- MetaCDN QoS Monitor:

    - Constantly probes storage providers and monitors data transfers to assess the performance of each provider.

- Load Redirector:

    - Controls content serving.
    - In charge of redirecting user content requests to the most suitable replica given the condition of the systems and the options specified during deployment.

- Connectors:

    - Interactions with storage clouds are managed by means of connectors,
    - which abstract away the different interfaces exposed by providers and present a uniform interface within the MetaCDN system.

SpotCloud

- SpotCloud is an online portal that implements a virtual marketplace where sellers and buyers can register and trade cloud computing services.
- The platform is a marketplace operating in the IaaS sector.
- Buyers look for compute capacity that meets the requirements of their applications, while sellers can make their infrastructure available to serve buyers.
- Comprehensive set of features expected for a virtual marketplace:

    - Detailed logging of all the buyers' transactions.
    - Full metering, billing for any capacity.
    - Full control over pricing and availability of capacity in the market.
    - Management of quotas and utilization levels for providers.

    - Federation management (many providers, many customers, but one platform).
    - Hybrid cloud support (internal and external resource management).
    - Full market administration and reporting.
    - Applications and pre-built appliances directories.

- By acting as an intermediary for trading compute and storage between consumers and service providers, SpotCloud provides both parties with added value.
- Allows users with available computing capacity to easily turn themselves into service providers by deploying the runtime environment required by SpotCloud on their infrastructure.
- For service consumers — acts as a market directory where they can browse and compare different IaaS service offerings and select the most appropriate solution for them.
- For service providers — constitutes an opportunity for advertising their offerings.
- SpotCloud was the first clearinghouse / marketplace for cloud computing services, launched by Enomaly.
- For cloud service providers, the SpotCloud marketplace platform provides an easy way to sell unused cloud capacity. Providers can clear out unused capacity and sell computing inventory that would otherwise go unsold — enabling increased utilization and revenue without undermining their standard pricing.
- SpotCloud is not only an enabler for IaaS providers and resellers — it also includes complete bookkeeping of transactions associated with the use of resources.
- Users deposit credit on their SpotCloud account, and capacity sellers are paid following the usual pay-per-use model.
- SpotCloud retains a percentage of the amount billed to the user.
- By leveraging a uniform runtime environment and virtual machine management layer, it provides users with a vendor-lock-in-free solution, which might be strategic for specific applications.

5.2 Dockers and Containers, Microservices, Cloud Automation Tools & DevOps

Dockers and Containers

Traditional Virtualization

- Server is the physical server used to host multiple virtual machines.
- Host OS is the base machine such as Linux or Windows.
- Hypervisor is either VMware or Windows Hyper-V used to host virtual machines.
- One would then install multiple operating systems as virtual machines on top of the existing hypervisor as Guest OS.
- Applications would then be hosted on top of each Guest OS.

Docker — Architecture

- Docker engine is used to virtualize the guest operating system which earlier used to run in virtual machines.
- All the Apps now run as Docker containers.

Dockers

- Docker containers are analogous to physical containers used to store, package, and transport goods — but instead of tangible goods, they're containers for software applications.
- A container is an environment that runs an application that is not dependent on the operating system.
- A Docker container is a portable unit of software that contains:
    - The application,
    - Along with all its associated dependencies and configuration.

What is a Container?

- Each container runs isolated tasks.
- It cannot harm the host machine nor come in conflict with other apps running in separate containers.
- The kernel of the host operating system serves the needs of running different functions of an app, separated into containers.

Virtual Machines and Containers

- Virtual machines run guest operating systems — the OS layer in each box.
- Resulting disk image and application state is an entanglement of:
    - OS settings,
    - System-installed dependencies,
    - OS security patches.
- Containers can share a single kernel; the only information that needs to be in a container image is the executable and its package dependencies, which never need to be installed on the host system.
- These processes run like native processes and can be managed individually.
- Because they contain all their dependencies, there is no configuration entanglement — a containerized app "runs anywhere."

Definition of Docker (Wikipedia)

"An open-source project that automates the deployment of software applications inside containers by providing an additional layer of abstraction and automation of OS-level virtualization on Linux."

Docker vs Containers

- Docker is a software platform that packages software into containers.
- Docker images are read-only templates that contain instructions for creating a container.

- A container in Docker is an isolated virtual environment that runs independently. It packs all components required to run an application — dependencies, libraries, and code.
- Apartment analogy:
    - Think of a Docker container as an apartment unit within a building. It contains all necessary facilities for the tenant to live independently of their neighbors — runtime, libraries, code.
    - Apartment ↔ Container.
    - Tenant ↔ Application.
    - If Docker containers are apartment units, Docker images serve as the floor plan — specifying components required to build the room and furniture needed for the tenant to live comfortably.

Dockerfile

- A Dockerfile is a text-based document used to create a container image.
- It provides instructions to the image builder on the commands to run, files to copy, startup command, and more.
- A Dockerfile is simply a text-based file with no file extension that contains a script of instructions.
- Docker uses this script to build a container image.
- A Dockerfile typically follows these steps:
    - Determine your base image,
    - Install application dependencies,
    - Copy in any relevant source code and/or binaries,
    - Configure the final image.
- Using a Dockerfile to build a Docker Image → in turn to run a Docker Container.

Containers — Logical Packaging

- Containers offer a logical packaging mechanism in which applications can be abstracted from the environment in which they actually run.
- This decoupling allows container-based applications to be deployed easily and consistently — regardless of whether the target environment is a private data center, the public cloud, or a developer's personal laptop.
- Gives developers the ability to create predictable environments that are isolated from the rest of the applications and can be run anywhere.

Docker Commands

- Set up Docker on Mac, Linux, and Windows.
- Test your Docker installation by running the appropriate test command.
- Run a Busybox container to get a taste of the docker run command.
- The pull command fetches the busybox image from the Docker registry and saves it to the system.
- Notes on permissions:
    - Depending on installation, you might see a permission denied error.
    - On Mac — make sure the Docker engine is running.

    - On Linux — prefix docker commands with sudo.

- Use the docker images command to see a list of all images on your system.

docker run

- Run a Docker container based on an image using docker run.
- When you call run, the Docker client finds the image (e.g., busybox), loads up the container, and runs a command in that container.
- If no command is provided (e.g., docker run busybox), the container boots up, runs an empty command, and then exits.
- Running echo in the busybox container outputs the message and the container then exits — happens very quickly. Imagine booting up a VM, running a command, and killing it — this is why containers are fast.
- Running docker run with the -it flags attaches you to an interactive tty in the container, where you can run as many commands as you want.

docker ps

- docker ps shows all containers that are currently running.
- If no containers are running, you see a blank line.
- Shows only running containers; lists containers that are currently active.
- Stopped or exited containers are not shown.

docker ps -a

- Shows all containers:

    - Running
    - Stopped
    - Exited
    - Created but never started

- The STATUS column shows that these containers exited a few minutes ago.

docker rm

- Talks about deleting containers.
- After exiting, remnants of the container are still visible via docker ps -a.
- Stray containers eat up disk space — clean them up once done.
- docker rm <container_id> deletes specific container(s); on deletion, the IDs are echoed back.
- To delete many at once:

    - Use docker rm with -q (returns only numeric IDs) and -f (filters output based on conditions).
    - Deletes all containers that have a status of exited.

docker prune

- In later versions of Docker, the docker container prune command can be used to achieve the same effect — clearing out exited containers.

Docker Terminology

- Docker Hub:
  - A registry of Docker images. Think of the registry as a directory of all available Docker images.
  - You can host your own Docker registries and use them for pulling images.
- Images:
  - The blueprints of an application which form the basis of containers.
  - Used the docker pull command to download the busybox image.
- Containers:
  - Created from Docker images and run the actual application.
  - Created using docker run (e.g., from the busybox image).
  - A list of running containers can be seen using the docker ps command.
- Docker Daemon:
  - The background service running on the host that manages building, running, and distributing Docker containers.
  - The daemon runs in the operating system; clients talk to it.
- Docker Client:
  - The command-line tool that allows the user to interact with the daemon.
  - Other forms of clients include GUIs such as Kitematic.

Microservices

Introduction

- Microservices are an architectural and organizational approach to software development.
- Software is composed of small independent services that communicate over well-defined APIs.
- These services are owned by small, self-contained teams.
- Microservices architectures make applications easier to scale and faster to develop, enabling innovation and accelerating time-to-market for new features. — Amazon AWS
- Also known as the microservice architecture — an architectural style that structures an application as a collection of services that are:
  - Independently deployable,
  - Loosely coupled,

- Organized around business capabilities,
- Owned by a small team.

“Do One Thing and Do It Well” — The Motto

- Projects considered “microservices” follow Ken Thompson’s Unix philosophy: "Do one thing and do it well."
- Summarized as: focus on one task, and perfect it.
- This statement is not just relevant to programming, but also describes the functionality of individual microservices.

Relation to SOA

- Microservice architecture is basically a further development of service-oriented architecture (SOA): small services also play a role in this architectural pattern.
- The microservice architecture enables an organization to deliver large, complex applications:
  - Rapidly,
  - Frequently,
  - Reliably and
  - Sustainably — a necessity for competing and winning in today’s world.

Monolithic vs. Microservices Architecture

Monolithic

- All processes are tightly coupled and run as a single service.
- If one process of the application experiences a spike in demand, the entire architecture must be scaled.
- Adding or improving features becomes more complex as the code base grows. This complexity limits experimentation and makes it difficult to implement new ideas.
- Monolithic architectures add risk for application availability because many dependent and tightly coupled processes increase the impact of a single process failure.
- Traditional program development works according to monolith principles: all tasks are undertaken in one large project. All individual services access a large database and are output through a user interface.

Microservices

- An application is built as independent components that run each application process as a service.
- These services communicate via well-defined interfaces using lightweight APIs.
- Services are built for business capabilities and each service performs a single function.
- Because they run independently, each service can be updated, deployed, and scaled to meet demand for specific functions of an application.
- While the monolith tries to combine everything in one application, microservices are only responsible for one task and run independently of each other.

- The microservices approach is based on modules: each microservice is responsible for completing a simple task.
- Each work process is vastly different, as are often the results.
- Team organization differs:
    - In microservices, each team is responsible for their own microservice.
    - Teams are often organized according to the technology they deal with — one for databases, one for individual services, one for UI design.
    - Other groups are responsible for publishing updates, maintenance, and analysis.
- All teams in a monolith are interdependent; dependencies in microservice architecture should be avoided as much as possible.

Characteristics of Microservices

Autonomous

- Each component service can be developed, deployed, operated, and scaled without affecting the functioning of other services.
- Services do not need to share any of their code or implementation with other services.
- Any communication between individual components happens via well-defined APIs.

Specialized

- Each service is designed for a set of capabilities and focuses on solving a specific problem.
- If developers contribute more code to a service over time and it becomes complex, it can be broken into smaller services.

Benefits of Microservices

Agility

- Foster an organization of small, independent teams that take ownership of their services.
- Teams act within a small and well-understood context, and are empowered to work more independently and quickly. This shortens development cycle times.
- You benefit significantly from the aggregate throughput of the organization.

Flexible Scaling

- Allow each service to be independently scaled to meet demand for the application feature it supports.
- Enables teams to right-size infrastructure needs, accurately measure the cost of a feature, and maintain availability if a service experiences a spike in demand.

Easy Deployment

- Enable continuous integration and continuous delivery, making it easy to try new ideas and roll back if something doesn't work.
- Low cost of failure enables experimentation, makes it easier to update code, and accelerates time-to-market for new features.

Technological Freedom

- Microservices architectures don't follow a "one size fits all" approach.
- Teams have the freedom to choose the best tool to solve their specific problems.
- Teams building microservices can choose the best tool for each job.

Reusable Code

- Dividing software into small, well-defined modules enables teams to use functions for multiple purposes.
- A service written for a certain function can be used as a building block for another feature.
- Allows an application to bootstrap off itself — developers can create new capabilities without writing code from scratch.

Resilience

- Service independence increases an application's resistance to failure.
- In monolithic architectures, if a single component fails, it can cause the entire application to fail.
- With microservices, applications handle total service failure by degrading functionality — not crashing the entire application.

Working with Microservices — 3 Examples

- Microservice architecture has now found its way into large company systems.
- Companies have subsequently been able to fix certain problems or optimize processes.
- Some were already using modular systems even when there was no term for them.
- Examples of large companies with established monolithic systems changing to a microservice model:
    - Netflix
    - Spotify
    - eBay
    - Other major IT companies like Google and Amazon also work like this.

Netflix

- Like many other companies, Netflix used to be based on a monolithic system (back when Netflix was not an online streaming service but only sent DVDs through the mail).
- In 2008, an error in a database caused the entire service to fail for four days.
- The decision was taken to break up the old system and split it into microservices. The result:
    - The company was able to make live changes much faster, and

    - Repairs were carried out much more quickly.

- Since the Netflix system is enormously extensive, a separate program was developed to organize the individual microservices among themselves: Conductor.
- Conductor grants Netflix central control to pause, restart, or scale their microservices.

Spotify

- The audio streaming service Spotify also relies on microservices.
- Spotify's main daily development challenge is keeping ahead of strong competition. The audio streaming services market has some of the largest IT companies — Amazon, Apple, Google.
- Due to the increasing number of users, Spotify developers must constantly meet higher demands and comply with certain business rules (like licensing rights).
- Microservices allow Spotify to react quickly to new developments competitors might make and publish their own developments faster — forcing competitors to react in turn.
- Example: Spotify's feature that recommends suggestions to users when they type in a search term is a self-contained microservice that has its own dedicated team working on it.
- Spotify benefits from the robust nature of microservice architecture: if a single microservice fails, it does not mean the entire product becomes unusable.
- There are more than 800 microservices active within Spotify, and they use Java for a large part of these microservices.
- This is not because microservices cannot be written in different languages — but rather has to do with work processes: developers constantly move from one team to another, and it is easier when everyone uses the same language.

eBay

- Like many other large systems, the eBay sales platform began as a monolith.
- eBay had 3.4 million lines of code in just one file.
- The company decided to break up the monolith and develop Java microservices instead.
- Individual eBay services also use REST to communicate with one another.

Inference

- The fact that eBay and other companies have successfully gone from a monolithic to microservice architecture is a clear sign of the benefits of a more modern approach.
- While the monolith structure is perfectly sufficient in the early days of a website (with a small number of active users and a manageable range of products), it can become growth-inhibiting when demands start to increase.

Cloud Automation Tools and DevOps Concepts

Note: This topic is part of the syllabus for Module 5.2. Below are concise notes covering the standard concepts associated with this topic.

DevOps — Definition

- DevOps is a combination of cultural philosophies, practices, and tools that increases an organization's ability to deliver applications and services at high velocity.
- It bridges the gap between Development (Dev) and Operations (Ops) teams to enable faster, more reliable software delivery.
- DevOps emphasizes automation, continuous integration, continuous delivery, monitoring, and collaboration.

Key DevOps Principles

- Continuous Integration (CI) — developers frequently merge code changes into a central repository, after which automated builds and tests run.
- Continuous Delivery (CD) — code changes are automatically built, tested, and prepared for release to production.
- Continuous Deployment — every change that passes automated tests is automatically deployed to production.
- Infrastructure as Code (IaC) — managing and provisioning infrastructure through machine-readable definition files rather than manual configuration.
- Monitoring and Logging — continuous tracking of application and infrastructure performance to detect issues proactively.
- Microservices — using small, independent services that can be developed and deployed independently.
- Collaboration and Communication — breaking down silos between development, operations, and other stakeholders.

Benefits of DevOps

- Speed — move at high velocity to innovate for customers and adapt to changing markets.
- Rapid Delivery — increase the frequency and pace of releases to innovate and improve faster.
- Reliability — ensure quality of application updates and infrastructure changes.
- Scalability — operate and manage infrastructure and development processes at scale.
- Improved Collaboration — between development and operations teams.
- Security — IaC + policy as code enable retaining control and compliance at scale.

DevOps Lifecycle (Phases)

- Plan — agile planning, backlog grooming, sprint planning.
- Code — version-controlled source code, code review.
- Build — compile, package, run unit tests.
- Test — automated functional, integration, and performance testing.
- Release — package versioned artifacts and prepare for deployment.
- Deploy — push to production using CI/CD pipelines and IaC.
- Operate — manage and maintain running services in production.
- Monitor — observability, logging, alerting, feedback into the next plan cycle.

Cloud Automation — Overview

- Cloud automation is the use of tools and processes to perform tasks in the cloud with minimal human intervention.
- Reduces manual effort, errors, and time required to provision, configure, manage, and de-provision cloud resources.
- Common areas of cloud automation:
  - Resource provisioning (servers, storage, networks).
  - Configuration management.
  - Auto-scaling and load balancing.
  - Backups and disaster recovery.
  - Security and compliance enforcement.
  - Cost optimization (shutting down idle resources).

Popular Cloud Automation & DevOps Tools

- Source Code Management:
  - Git, GitHub, GitLab, Bitbucket
- CI/CD:
  - Jenkins, GitLab CI/CD, CircleCI, Travis CI, GitHub Actions, AWS CodePipeline, Azure DevOps
- Configuration Management:
  - Ansible, Chef, Puppet, SaltStack
- Infrastructure as Code (IaC):
  - Terraform, AWS CloudFormation, Azure Resource Manager (ARM), Pulumi
- Containerization:
  - Docker
- Container Orchestration:
  - Kubernetes, Docker Swarm, Amazon ECS / EKS, Azure AKS, Google GKE
- Monitoring & Logging:
  - Prometheus, Grafana, Nagios, ELK Stack (Elasticsearch, Logstash, Kibana), Splunk, CloudWatch, Datadog
- Collaboration:
  - Slack, Microsoft Teams, Jira, Confluence

Relationship between Cloud, Containers, Microservices, and DevOps

- Cloud provides scalable, on-demand infrastructure.
- Containers (e.g., Docker) provide a lightweight way to package and run applications uniformly across environments.
- Microservices decompose monolithic applications into small, independent services that fit naturally into containers.
- DevOps and CI/CD pipelines enable automated build, test, and deployment of these microservices into the cloud.
- Together, they enable rapid, reliable, and scalable cloud-native application delivery.